DhakaPath: Live Urban Transit & Route Intelligence

Official Hackathon Problem Statement | INIF 2026

Theme:	Smart City	**Duration:**	5 Hours 40 Minutes
Platform:	Open Tech Stack	**Presentation:**	4–5 Mins Demo + 2 Mins Q&A

Problem Statement

Standard navigation platforms are built primarily for private vehicles and cities with fixed, predictable transit schedules. In Dhaka, daily travel relies on a complex, fragmented network of local buses, the MRT-6 Metro Rail, legunas, and connector rickshaws. Existing map tools fail to account for student half-fares, rickshaw fare surges during bad weather, or sudden route disruptions caused by monsoon waterlogging and intersection blockades. Commuters are left depending on scattered social media posts that become outdated within minutes.

Your challenge is to design and build an intelligent multi-modal transit, fare calculation, and live street-alert platform tailored for Dhaka. The system must organize transit routes and live street reports, support natural language trip queries, calculate accurate fares and detours, validate crowd-sourced data, and provide a conversational interface for commuters.

Technical & Fair-Use Constraints (100% Free Stack)

To ensure a level playing field for all participants, paid APIs or credit-card-gated services are not required or rewarded:

- **Mapping & Visualization:** Paid map services (such as Google Maps) must not be used. Teams may use **Leaflet.js with OpenStreetMap** (free, no API key required) or build an **interactive node-graph / schematic transit map** covering 12–15 major Dhaka hubs (e.g., Motijheel, Banasree, Malibagh, Shahbagh, Farmgate, Science Lab, Dhanmondi, Mohammadpur, Mirpur, Agargaon, Mohakhali, Badda, Uttara, and Jatrabari).
- **AI & Language Models:** Teams must use free-tier APIs or local models, such as the Google Gemini API (Google AI Studio free tier), Groq Cloud free tier, HuggingFace Serverless Inference, OpenRouter free models, or local instances via Ollama.
- **Media & Speech (Optional):** Browser-native APIs (such as the Web Speech API or HTML Canvas) or free open-source libraries should be used for any bonus features.

System Overview

The platform operates across two main system components:

- **Transit Network & Route Graph:** The underlying structure of Dhaka hubs, connectors, and baseline fares.
- **Live Street Alerts & Disruption Logs:** Active roadblocks, waterlogging, traffic severity, and user-submitted reports.

Admins manage the core route network and moderate reports, while commuters interact with the system through visual interfaces and an AI assistant.

Part 1: Transit & Street-Status Management (CMS)

Build a content and state management system where admins and users can maintain transit connections and live street disruptions, including:

- Public transport links (local bus routes, MRT-6 Metro stations, leguna routes, and connector rickshaw segments)
- Live street incident reports (waterlogging, road construction, severe gridlock, or intersection blockades)
- Boarding landmark notes and base fare charts

Requirements: Data must be organized under Transit Network or Street Alerts. Commuters must be able to browse active routes, hub statuses, and disruptions with appropriate metadata (transport mode, origin/destination hub, base BDT fare, student half-pass eligibility, and estimated travel time).

Part 2: Intelligent Multi-Constraint Trip Search

Implement an intelligent search system that allows commuters to find routes and check road conditions using natural language in English or Banglish.

- **Natural Language Parsing:** Support intent extraction beyond simple keywords (e.g., querying routes within a budget or avoiding specific congested intersections).
- **Multi-Leg Combinations:** Return valid multi-leg route options (such as rickshaw to metro, metro to walking leg) ranked by total cost, estimated duration, and number of transfers.
- **Bonus:** Include specific landmark boarding and transfer instructions for each leg of the journey.

Part 3: Dynamic Fare Estimation & Disruption Rerouting

The platform must generate accurate fare breakdowns and alternative route plans based on the underlying transit graph and active street conditions.

- **Multi-Modal Fare Breakdown:** Calculate total trip cost in BDT, comparing regular fares against student half-fares (accounting for the 50% student discount on buses, the 10 BDT minimum bus fare floor, and modes where student passes do not apply). Provide estimated connector fare ranges for non-metered segments (rickshaws or CNGs), indicating baseline rates versus rush-hour or weather surge rates.
- **Interactive Scenario Simulation:** Include simulation controls where users can trigger urban disruptions (such as heavy rain/waterlogging in specific zones, peak office rush hour, or a blocked intersection). The system must automatically recalculate affected routes, identify broken segments, and output a functional detour plan with updated time and cost estimates.

Part 4: Report Validation & Output Grounding

Since crowd-submitted traffic alerts can be inaccurate and AI models can hallucinate non-existent routes, design a validation mechanism for both user reports and AI-generated travel plans.

- **Route Grounding:** Verify AI-generated route recommendations against the Part 1 transit graph to ensure every suggested bus, station, and fare exists in the system.
- **Crowd Corroboration & Decay:** Allow users to confirm or mark alerts as resolved, while automatically downgrading or expiring older traffic reports that have not been recently re-verified.
- **Sanity Checks:** Apply rule-based filtering to flag unrealistic fare submissions or contradictory

incident reports before they impact route calculations.

Part 5: Conversational Commute Assistant

Provide a chat-based interface supporting English and Banglish through which users can access core system features:

- Search for multi-leg routes and compare fares.
- Submit a new live street disruption report through natural conversation.
- Request alternative routes when a specific hub is congested or blocked.
- Ask contextual follow-up questions about fare rules or alert reliability.

Note: The chat interface maintains conversational context and acts as an alternative interaction layer alongside the core visual application.

Expected Deliverables

- Working software prototype (deployed online or demonstrated locally)
- GitHub repository with source code developed during the session and setup instructions
- Explanation of system architecture, routing logic, and AI components
- Live demonstration during the 4–5 minute presentation

Bonus Tasks

1. **Signboard & Notice Digitization:** Allow users to upload a photo of a local bus signboard, printed fare chart, or handwritten route note to automatically extract structured route data into the platform.
2. **Audio/Visual Route Briefing:** Convert a selected multi-leg route and active street alerts into a concise audio briefing or downloadable offline visual summary card for commuters with low battery or poor connectivity.
3. **Commuter Ride-Share & Bot Coordination:** Integrate a community board where commuters at the same transit hub can coordinate to share a rickshaw or CNG and split the fare, supported by an automated bot that calculates per-person costs.